

# Argo: Efficient Importance Labeling for Enterprise Email Systems

Siddhant Ray\*  
University of Chicago

Ganesh Ananthanarayanan  
Microsoft

Kevin Chian  
Microsoft

Yan Guo  
Microsoft

Cristina St Hill  
Microsoft

Jack W. Stokes\*  
Independent

Victor Wang  
Microsoft

Junchen Jiang  
University of Chicago / TensorMesh

## Abstract

Email importance labeling has been critical for businesses and individuals. However, traditional keyword matching and user-defined rules require manual feature engineering and fail to scale or generalize. Recent LLMs such as GPT-4.1 provide high labeling quality, but enterprise-scale deployment remains prohibitively expensive. We develop Argo, an enterprise-email labeling framework that explores alternative labeling schemes matching GPT-4.1 quality at significantly lower cost, using an efficient profiler to search the cost-quality tradeoff space and identify cost-efficient solutions. Argo also includes an on-demand provisioning scheme to scale labeling with real-time load and minimize peak-load cost increases. Across 3 open-source email datasets, Argo achieves 148-167× inference cost reduction with negligible quality degradation and 20-640000× lower profiling costs, enabling practical, large-scale email labeling for enterprises.

**CCS Concepts:** • Information systems → Enterprise information systems; Enterprise resource planning.

**Keywords:** SLM cascades; Adaptive provisioning

## ACM Reference Format:

Siddhant Ray, Ganesh Ananthanarayanan, Kevin Chian, Yan Guo, Cristina St Hill, Jack W. Stokes, Victor Wang, and Junchen Jiang. 2026. Argo: Efficient Importance Labeling for Enterprise Email Systems. In *Practical Adoption Challenges of ML for Systems (PACMI '26)*, September 29–October 02, 2026, Prague, Czech Republic. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3843967.3844682>

\*Work done while at Microsoft.



This work is licensed under a Creative Commons Attribution 4.0 International License.

PACMI '26, Prague, Czech Republic

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2998-0/2026/09

<https://doi.org/10.1145/3843967.3844682>

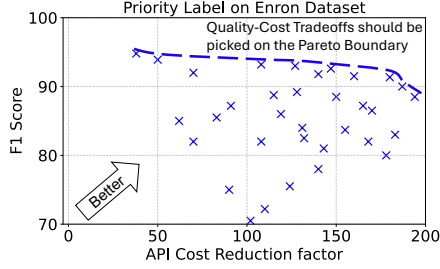
## 1 Introduction

Email importance labeling streamlines personal and organizational workflows by directing user attention toward urgent tasks and time-sensitive messages [2, 10]. It enables cleaner task management by organizing action items and related data into appropriate folders [4]. While email labeling is a long-studied problem, prior systems rely heavily on manually engineered features to approximate message importance [1, 2, 10, 21, 23]. We revisit enterprise-level email importance labeling in the era of highly expressive large language models (LLMs), which are a natural fit for this task. LLMs provide contextual understanding and operate directly on natural-language email content without manual featurization, allowing LLMs to achieve substantially higher-quality labeling than older feature-engineered systems.

A reliable and well-integrated labeling feature strengthens an enterprise-email client's competitiveness, increases user adoption and supports revenue growth [13]. We carry out this work in collaboration with Outlook, a leading email service aiming to deploy such solutions in their product.

However, enterprise-level email workloads have extremely high volumes, with 376 billion emails exchanged per day in 2025 [18]. While LLMs like GPT4.1[5] offer superior labeling quality, running labeling with an LLM at this scale for every email leads to *prohibitive running costs*. Leading email providers like Outlook, while desiring such labeling quality, are balking at the cost of labeling at this scale (e.g., GPT4.1 leads to labeling costs of \$5-10 billion per month at current email volumes). We ask the question, how do we build an *efficient system* which achieves the best tradeoffs for email labeling by intelligently balancing quality and cost? Further, how do we *efficiently* provision resources to allow deployed enterprise-level email systems label under real-time variable load, without significantly increasing costs?

Lower-cost alternative systems to LLMs for email labeling, either use cascades of Small Language Models (SLMs) [7, 12, 17] or trained classifiers using text embeddings [3, 16, 20]. However, these systems have *multiple knobs* (e.g., which SLMs to use in the cascade etc.) which leads to an *extremely large search space* (> 100M configurations, more in §2.2).



**Figure 1.** Varying *only* the confidence thresholds for the SLM Cascade already leads to a *large* tradeoff space

To deploy such labeling systems for achieving best cost-quality tradeoffs, we need an *efficient* profiler to navigate the search space and manage inference cost. We introduce Argo, which has (a) an efficient profiler to select labeling methods achieving best quality-cost tradeoffs and identify a *Balanced* operating point and (b) a resource-provisioning scheme to minimize cost increases under peak email load.

Unlike several traditional profiler-based systems, messaging workloads like emails, do not have clear spatial or temporal insights to be utilized for profiling (e.g., several video analytics profiling systems [8, 9, 22, 24] relied on such properties). Argo’s profiler identifies a *novel insight*, the characteristics of the labels can be used as a signal to choose the labeling method. This allows us to make offline-profiled static choices for parameters at runtime. Offline profiling on a calibration set of emails allows Argo to decide the labeling method at runtime; a dynamic choice between a cascade of SLMs and a lightweight embedding classifier, along with all necessary parameters to construct the SLM cascade.

Email labeling at enterprise scale also requires an efficient system to manage resources under peak email load. Outlook is provisioned to use SLMs via *model-as-service* at a certain *capacity* and is charged a penalty for exceeding it. Argo develops a greedy resource provisioning algorithm for the SLM cascade, to balance between further provisioning SLM instances which are at capacity and using other under-capacity SLMs in the cascade, to minimize cost increases during load.

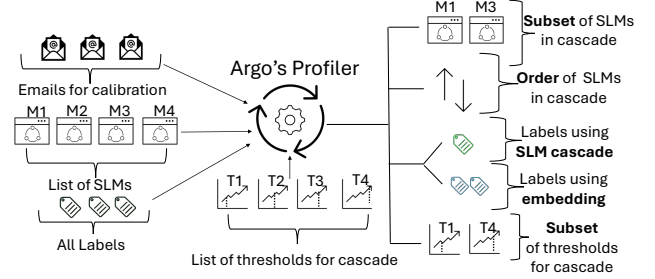
We make the following contributions and enable Outlook to deploy these labeling techniques in their email client -

- Argo is the first system for *cost-efficient* labeling of email enterprise workloads with near LLM labeling accuracy at 148-167 $\times$  lower inference costs.
- Argo efficiently searches the quality-cost tradeoff space for email labeling and chooses a *Balanced* point with 20-640000 $\times$  lower profiling costs with no quality drop.
- Argo provides a resource provisioning scheme to minimize cost-increase of labeling under increased load by 2.2-3.8 $\times$ .

## 2 The tradeoff space for email labeling

### 2.1 What do email labels look like for Outlook?

- *Priority* - This provides an overall importance ranking of the email on a discrete scale from 1 (lowest) to 5 (highest).



**Figure 2.** Argo performs offline profiling to determine knob values and decisions to be used for email labeling

- *Needs Reply* - This binary label determines if an email requires a reply or not.
- *Is Urgent* - This binary label measures the time-sensitivity and how quickly should the email be read by the receiver.
- *Needs Action* - This binary label decides if the receiver needs to carry out an explicit task (e.g., access approval).
- *Needs Scheduling* - This binary label determines if the email needs a time-sensitive follow up meeting or event.

To establish ground-truth labels, we use GPT-4.1, the reference model deployed in Outlook, to label each email. We additionally validate these labels through human review by team members, confirming that their quality meets Outlook’s deployment requirements. We therefore use the GPT-4.1-generated labels as ground truth and evaluate our system by measuring its labeling performance relative to these labels.

### 2.2 Optimal tradeoffs needs a *smart* system

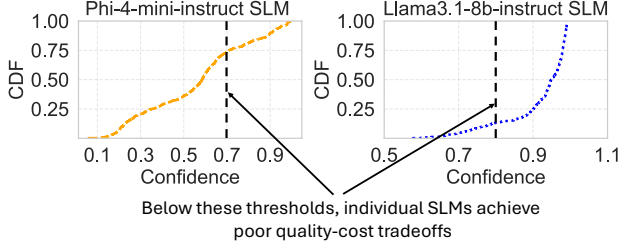
Offline profiling over a set of calibration emails is an effective way to assign criteria for labeling (e.g., when to choose the SLM cascade) However, naive profiling leads to prohibitive costs due to the large combinatorial tradeoff space, as there are several knobs which must be explored. The important knobs are *number*, *order* and *choice* of SLMs in cascade, *confidence thresholds* within the cascade, and *how* and *when* to choose between embedding classifier or the cascade.

Conducting an exhaustive sweep over all combinations of knobs *prohibitively* increases the cost of the profiler. If we vary *only* the confidence threshold for the SLM cascade in the range 80 – 100%, while keeping all other knobs fixed, the cost-quality tradeoff space varies significantly varying a *single knob* in a small neighborhood (Figure 1). It becomes *essential* to reduce the space of profiling and build a cost-efficient profiler to find the best solution the tradeoff space.

## 3 Argo: A System for Cost-Efficient Enterprise Email Labeling

We present Argo (Figure 2), an email importance labeling system for enterprise scale. Argo has two main components-

- An *efficient profiler* which runs offline profiling to determine the optimal quality-cost tradeoffs for email labeling.



**Figure 3.** Log-prob confidence distribution for two open-source SLMs on a calibration set from the Enron Dataset

- An *on-demand* resource provisioning scheme to scale Argo to minimize the cost increases during peak load.

### 3.1 How and over what do we profile?

- *Calibration set* - This is the set of emails used to offline profile and decide factors like choice of cascade’s SLMs.
- *Email Labels* - This is the list of labels with their requirements (e.g., nature of the label) which is used to determine the best labeling method to assign values at runtime.
- *SLMs and embedding models* - This contains the whole list of SLMs and text embedding models which can be used.
- *Range of confidence thresholds* - This provides lower and upper bounds of confidence thresholds, from which the SLM cascade chooses values for optimal quality-cost tradeoffs.

Argo’s profiler compares the quality-cost tradeoff of generating a label using a knob value compared to the same label generated by the LLM baseline. This gives us *Pareto-efficient* choices for knob values to label the emails.

Finally to generate labels, Argo takes as input both the email content and associated metadata such as the sender, recipients, and CC list. Some metadata fields are universally available (e.g., sender name and CC list), while others, such as the sender’s organizational level, are incorporated when available. This metadata can provide useful contextual signals and improve labeling quality in some cases. However, our experimental analysis showed that the email body remained the primary source for producing high-quality labels, due to higher context information present.

### 3.2 Selecting knob values with Argo’s profiler

Argo establishes that label characteristics as a signal to choose the labeling method at runtime. Based on extensive profiling, Argo *decides* that an SLM cascade is best if the label is multi-class and a trained embedding classifier suffices, if the label is binary. Argo’s profiler selects the subset of SLMs and confidence thresholds for the cascade, ordering SLMs by increasing model size to *minimize* cost. Given an email, Argo sends it to the first SLM and keeps the label if confidence exceeds the chosen threshold; else, it moves up the cascade.

The embedding classifier is 3-layer neural network and is trained using embeddings of the emails from the input

---

### Algorithm 1: Greedy Allocation With Instance Penalty and Strict Running-Cost Ordering

---

**Input:** Total demand  $D$ , number of requests  $r$ , instance capacity  $C$ , instance base-costs  $c_i$ , penalty  $p \geq 1$ , running costs per request  $z_1 < z_2 < \dots < z_m$ , initial steady-state requests  $k_i$ , initial instances  $n_i$ , number of models  $m$  in the cascade

**Output:** Updated  $n_i, k_i$ , TotalCost

```

1  $D_{\text{req}} \leftarrow D/r$ ;
2 for each new request do
3   for  $i \leftarrow 1$  to  $m$  do
4     if  $k_i D_{\text{req}} < n_i C$  then
5        $k_i \leftarrow k_i + 1$ ; // Model  $i$  has free instance capacity
6       break;
7     else
8        $\Delta \text{InstCost} \leftarrow c_i p^{n_i}$ ; // Provisioning a new instance
9        $\Delta \text{RunCost} \leftarrow z_{i+1} - z_i$ ; // Cost of pushing to next model
10      if  $\Delta \text{InstCost} < \Delta \text{RunCost}$  then
11         $n_i \leftarrow n_i + 1$ ; // Cheaper to add an instance here
12         $k_i \leftarrow k_i + 1$ ; // Serve request on the same model
13        break;
14      else
15        continue; // Try the next more expensive model
16 InstanceCost  $\leftarrow \sum_{i=1}^m c_i \frac{p^{n_i} - 1}{p - 1}$ ;
17 RunCost  $\leftarrow \sum_{i=1}^m k_i z_i$ ;
18 TotalCost  $\leftarrow$  InstanceCost + RunCost;
19 return  $n_i, k_i$ , TotalCost

```

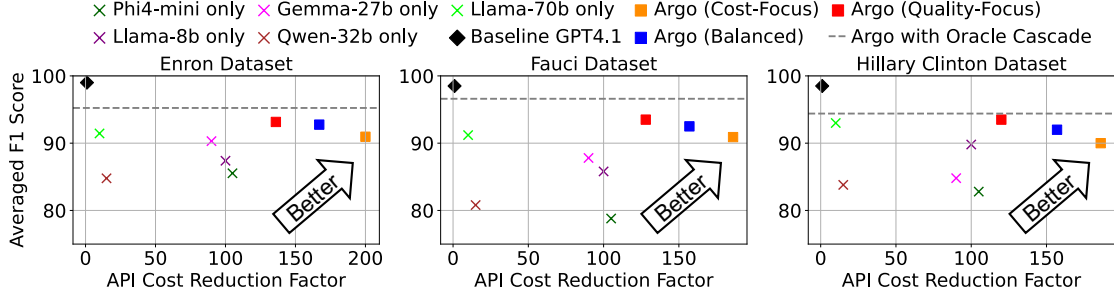
---

calibration set. The profiler decides the optimal choice of embedding model to be used. The generated embeddings are reused across all binary labels. The classifier is *offloaded* to the CPU for training and inference, which saves GPU costs.

Argo’s objective is to perform email labeling while balancing cost and quality. Argo balances the tradeoff by choosing a point on the *Pareto Front* which *maximizes* sum of the *normalized* generation quality (wrt. to baseline LLM) and the cost reduction (wrt. to cheapest model). By default, the *Balanced* policy is chosen, however Argo allows operators to specify weights for each dimension for custom tradeoffs.

### 3.3 How do we manage the cost of profiling?

**Incrementally building the calibration email set:** A natural challenge in Argo’s design is to control the cost of the offline profiling to realize cost savings during inference. The size of the offline calibration set of emails for profiling is an essential knob, if too small, the profiler may not generalize while if the size is too large, the profiling may waste resources, without providing additional gain. Argo tackles this by incrementally building the calibration set, with respect to a holdout *validation set*. We start with a small calibration set, profile on it and test on the validation set. We continue increasing the size of the calibration set till Argo finds points on the quality-cost *Pareto Front* on the validation set.



**Figure 4.** Argo finds multiple Pareto Efficient cost-quality tradeoff points for the SLM cascade and chooses the *balance* point (blue square) which achieves 148-167 $\times$  cost reduction with comparable quality

**Pruning the space of profiling knobs:** Argo reduces profiler cost by exploiting empirically observed independence among profiling knobs. The *first* observation is that optimal SLMs in the cascade can be chosen independently based on each SLM’s cost-quality tradeoff space, as SLMs with better individual tradeoffs form better cascades. *Second*, ordering SLMs by increasing size minimizes cost, since cheaper models are used more frequently. Finally, Argo performs an offline sweep over confidence thresholds for the selected SLMs and chooses threshold values on the *Pareto Front*, without requiring an exhaustive search. When individual SLMs (Figure 3) consistently achieve poor quality below certain confidence values, Argo discards those values from the search space.

### 3.4 On-demand provisioning to minimize email labeling costs under increasing load

Argo also builds a provisioning scheme to scale and manage the SLM cascade with increasing email load, to ensure minimal cost increases. Outlook uses SLMs for labeling via *model-as-a-service* hosted APIs and is billed per real-time requests made. We setup the cost model for the SLM cascade with two costs based on Outlook’s setup for deployment.

- *Per-request usage cost* - This is the inference cost of serving a request on a given model. Requests are sent to a model instance and billing is based on real-time usage.
- *Model instance provisioning cost* - This is the cost of creating an SLM instance. Provisioning under available capacity incurs only the base cost; once capacity is reached, each additional SLM instance incurs a penalty factor  $p$  to ensure fair sharing with other services.

Algorithm 1 minimizes cost by choosing the smallest *marginal cost increase* at each cost-increasing decision. In lines 4–6, it assigns requests to the current model instance while capacity remains. Once the email arrival rate increases and instance capacity is reached, lines 8–15 compare the marginal cost of provisioning a new instance with penalty  $p$  against moving up to an SLM with higher *per-request cost*.

Based on whichever adds *lower cost*, Argo adds a new instance or moves up the cascade, continuing this process for subsequent requests. Algorithm 1 is *optimal* for minimizing inference cost increases as (a) per-request costs increase monotonically up the cascade and (b) every cost-increase decision minimizes the *marginal cost* increase. As costs are additive, minimizing each increase minimizes total cost.

## 4 Evaluation

### 4.1 Setup. Datasets and Baselines

Argo’s profiler chooses 5 open-source SLMs for the cascade and one embedding model for the classifier. The API cost is measured using token cost [14] of running the models on Azure AI Foundry. For local mode, we use upto 4 NVIDIA A100 GPUs hosted on Azure to run the SLM cascade with vLLM [11]. The *model-as-a-service* mode is run as a API on Azure AI Foundry. The embedding classifier is trained and run on an Azure CPU server.

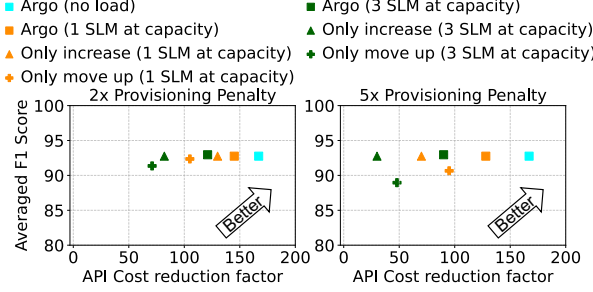
We evaluate Argo’s performance on three public email datasets: the Enron [6], Fauci [15], and Hillary Clinton [19] email corpora. For evaluation, classifier training and profiling, all emails are labeled with the baseline GPT4.1 LLM.

**Inference Baselines:** In Figure 4, we compare Argo with GPT-4.1, individual SLMs, and an oracle SLM cascade (as a quality upper bound). GPT-4.1 generates reference labels for measuring F1-score and API Cost Reduction Factor, while the oracle selects the least expensive SLM that matches GPT-4.1, defaulting to the most expensive SLM when none agree.

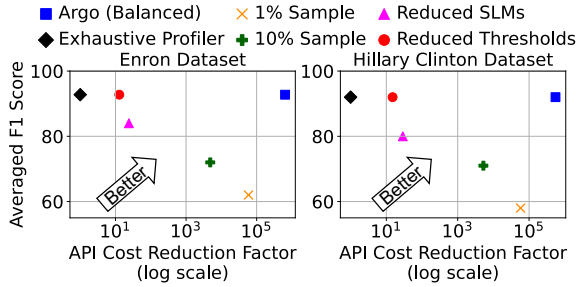
**Profiler Baselines:** We compare against exhaustive profiling over all configurations on the full calibration set, as well as variants using random 1% and 10% samples. We also evaluate reduced search spaces that either retain only three SLMs with the best individual quality-cost tradeoffs or profile only thresholds above the median value on the full calibration set.

### 4.2 Measuring performance gains with Argo

**Cost-reduction at comparable quality:** Figure 4 shows Argo’s quality-cost tradeoffs for email labeling across three



**Figure 5.** Argo resource provisioning is 2.2-3.8 $\times$  lower cost than baselines across multiple penalty factors and capacity bottlenecks (results on Enron Dataset)



**Figure 6.** Argo achieves 20-640000 $\times$  lower profiling costs compared to all baselines maintaining same or higher quality

open-source email datasets, reporting average F1-score across labels. By default, Argo selects *Balanced*, achieving a 148-167 $\times$  API cost reduction over the GPT-4.1 baseline at comparable quality, while outperforming every individual SLM in the cascade with 5-15% higher quality and 1.67-17 $\times$  lower API cost. We also show *Cost Focus* and *Quality-Focus* operating points on the Pareto front, achieving 186-200 $\times$  and 120-136 $\times$  API cost reductions respectively, allowing Outlook to meet different requirements. We validate quality tolerance providing values acceptable to Outlook’s business needs.

**Cost-Efficient scaling under peak load:** We simulate load by increasing the arrival rate over evaluation emails. We maintain a *request-queue* for each SLM instance and *at-capacity* is reached when the queue is full. The initial instance allocation is based on each SLM’s usage fraction from profiling. Figure 5 evaluates Argo’s provisioning under increasing load on the Enron dataset. We consider bottleneck capacities for 1 SLM (orange) and 3 SLMs (green) under penalty factors of 2 $\times$  and 5 $\times$ . Across penalty factors and capacity bottlenecks, Argo achieves 2.2-3.8 $\times$  lower cost increase than both: (a) always increasing instances on demand and (b) always moving up the cascade for residual capacity.

**Argo enables cheaper profiling without quality drop:** Figure 6 compares Argo’s profiler cost against profiler baselines and measures quality against GPT-4.1 labels. Argo matches exhaustive profiling quality while being 640000 $\times$  cheaper. In

contrast, the 1% and 10% sample baselines are 31000 $\times$  and 4850 $\times$  cheaper than exhaustive but have 33% and 21% lower quality respectively. Reduced Cascade has a quality drop of 12% and is only 24-29 $\times$  cheaper, while Reduced Threshold matches exhaustive quality but is only 12-15 $\times$  cheaper.

## 5 Conclusion

We present Argo, a system for low cost email-labeling at enterprise scale. It uses efficient profiling and inference, alongside intelligent provisioning to minimize costs at peak load. Argo is designed to meet the production and adoption needs for Outlook’s deployment in their email client.

## Acknowledgment

We thank all the anonymous reviewers for their insightful feedback and suggestions. The project was funded by as a research intern project by Microsoft by the Outlook Data Team and the Networking Research Group.

## References

- [1] Sakhar Alkhereyf and Owen Rambow. Work hard, play hard: Email classification on the avocado and enron corpora. In *Proceedings of TextGraphs-11: the Workshop on Graph-based Methods for Natural Language Processing*, pages 57–65. Association for Computational Linguistics, 2017.
- [2] Ron Bekkerman. Automatic categorization of email into folders: Benchmark experiments on enron and sri corpora.
- [3] Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. Supervised learning of universal sentence representations from natural language inference data. In Martha Palmer, Rebecca Hwa, and Sebastian Riedel, editors, *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, Copenhagen, Denmark, September 2017. Association for Computational Linguistics.
- [4] Nicolas Ducheneaut and Victoria Bellotti. E-mail as habitat: an exploration of embedded personal information management. *Interactions*, 8(5):30–38, September 2001.
- [5] OpenAI et al. Gpt-4 technical report, 2024.
- [6] Federal Energy Regulatory Commission. Enron email dataset. <https://www.cs.cmu.edu/~enron/>, 2004. Accessed 2025-12-09.
- [7] Neha Gupta, Harikrishna Narasimhan, Wittawat Jitkrittum, Ankit Singh Rawat, Aditya Krishna Menon, and Sanjiv Kumar. Language model cascades: Token-level uncertainty and beyond. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Representation Learning*, volume 2024, pages 4147–4180, 2024.
- [8] Kevin Hsieh, Ganesh Ananthanarayanan, Peter Bodik, Shivaram Venkataraman, Paramvir Bahl, Matthai Philipose, Phillip B. Gibbons, and Onur Mutlu. Focus: Querying large video datasets with low latency and low cost. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 269–286, Carlsbad, CA, October 2018. USENIX Association.
- [9] Junchen Jiang, Ganesh Ananthanarayanan, Peter Bodik, Siddhartha Sen, and Ion Stoica. Chameleon: scalable adaptation of video analytics. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, SIGCOMM ’18*, page 253–266, New York, NY, USA, 2018. Association for Computing Machinery.
- [10] Bryan Klimt and Yiming Yang. The enron corpus: A new dataset for email classification research. In *European Conference on Machine Learning (ECML)*, 2004.

- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery.
- [12] Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Wei Liu, Jian Luan, Xiwen Zhang, Nicholas D. Lane, and Mengwei Xu. Demystifying small language models for edge deployment. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14747–14764, Vienna, Austria, July 2025. Association for Computational Linguistics.
- [13] Wendy E. Mackay. More than just a communication system: Diversity in the use of electronic mail. In *Proceedings of the ACM Conference on Computer-Supported Cooperative Work (CSCW)*, pages 344–353. ACM, 1988.
- [14] OpenAI. Openai api pricing — text token costs. <https://platform.openai.com/docs/pricing>, 2025. Accessed December 9, 2025.
- [15] Hani Osman. Fauci emails dataset, 2021.
- [16] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, November 2019. Association for Computational Linguistics.
- [17] Duncan Soiffer, Steven Kolawole, and Virginia Smith. Semantic agreement enables efficient open-ended LLM cascades. In Saloni Potdar, Lina Rojas-Barahona, and Sebastien Montella, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 2499–2537, Suzhou (China), November 2025. Association for Computational Linguistics.
- [18] Statista Research Department. Number of e-mails per day worldwide 2018-2028, 2026. Accessed: July 7, 2026.
- [19] U.S. Department of State. Hillary clinton email archive. <https://wikileaks.org/clinton-emails/>, 2016. FOIA Release; Accessed 2025-12-09.
- [20] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Improving text embeddings with large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11897–11916, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [21] Wei Wang, Saghar Hosseini, Ahmed Awadallah, Paul Bennett, and Chris Quirk. Context-aware intent identification in email conversations. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1209–1218. ACM, 2019.
- [22] Mike Wong, Murali Ramanujam, Guha Balakrishnan, and Ravi Ne-travali. MadEye: Boosting live video analytics accuracy with adaptive camera configurations. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 549–568, Santa Clara, CA, April 2024. USENIX Association.
- [23] Liu Yang, Susan Dumais, Paul Bennett, and Ahmed Awadallah. Characterizing and predicting enterprise email reply behavior. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 505–514. ACM, 2017.
- [24] Haoyu Zhang, Ganesh Ananthanarayanan, Peter Bodik, Matthai Philipose, Paramvir Bahl, and Michael J. Freedman. Live video analytics at scale with approximation and Delay-Tolerance. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 377–392, Boston, MA, March 2017. USENIX Association.